AWS Trading Platformv2 added security features

Scope of Work

Exchange architecture, AI Guardian and dynamic hot wallet controls

Prepared by Dzoan Nguyen Tran | DevOps and Cloud Engineer | 12 November 2026

Problem 5: Fortify The Castle

Task

Take the architecture you designed in **Problem 1** and make it secure.

Go through it and integrate security into the design — not as a section bolted on at the end, but as part of the architecture itself. Show us where you changed the design, where you added something, and where you decided the risk was acceptable as it stood.

We are interested in your reasoning and your priorities: what you secure first, what you would defer, and what you would refuse to ship without.

Where you need facts you do not have, say what you would need, how you would get it, and what you assumed in the meantime.

Submission

Write your answer in `SOLUTION.md`. Include:

- The updated architecture, with your security changes marked
- Why each change is there, and what it protects against
- What you deliberately left out, and the trade-offs you accepted

Project summary and demo

This design proposes a resilient spot-trading platform on AWS, using EKS across Availability Zones, Aurora PostgreSQL, an AI Guardian and simulated compartmentalized wallets. Prometheus and Grafana support operations; cross-Region backups support disaster recovery. Service alternatives, scaling and cost tradeoffs are documented above.

Demo link: [Insert the deployed demo or public repository URL here]. A working demo has not yet been provided or verified.

The challenge requests an architecture diagram, service choices with alternatives and a scaling plan. Implementation, performance and recovery outcomes must be reported only after testing.

1 Challenge requirements

Architecture a highly available trading system with similar features to the Binance trading platform https://www.binance.com. This system will need to be resilient to failures, scalable, and cost-effective. You should design the architecture, choose the appropriate technologies, and explain your approach for maintaining high availability and scalability.

Due to time constraint, you will not be able to cover every feature on the reference platform, so choose some features that will help you demonstrate your mastery of cloud infrastructure.

Deliverables

Your submission should include:

- An overview diagram of the services used and what role they play in the system.
- Elaboration on why each cloud service is used and what are the alternatives considered.
- Plans for scaling when the product grows beyond your current setup.

You are limited to the following constraints for your plan.

Requirement	What the submission must demonstrate
Cloud provider	Use Amazon Web Services (AWS).
Trading platform	Design a system with selected features similar to a cryptocurrency trading platform.
Throughput	Support 500 requests per second.
Response time	Target p99 response time below 100 ms. Define the measurement conditions and explain how to test them.
Failure resilience	Explain how the system responds to component and infrastructure failures.
High availability	Explain how services remain available or recover after a failure.
Scalability	Explain how the platform can handle growth beyond the initial workload.
Cost-effectiveness	Select services and capacity that balance cost, availability and performance.

Added Security features task

Task

Take the architecture you designed in **Problem 1** and make it secure.

Go through it and integrate security into the design — not as a section bolted on at the end, but as part of the architecture itself. Show us where you changed the design, where you added something, and where you decided the risk was acceptable as it stood.

We are interested in your reasoning and your priorities: what you secure first, what you would defer, and what you would refuse to ship without.

Where you need facts you do not have, say what you would need, how you would get it, and what you assumed in the meantime.

Submission

Write your answer in `SOLUTION.md`. Include:

- The updated architecture, with your security changes marked
- Why each change is there, and what it protects against
- What you deliberately left out, and the trade-offs you accepted

Selected features and assumptions

The Guardian and hot-wallet controls are our proposed additions; they are not explicitly required by the challenge. The wallet feature is scoped as a simulation or testnet demonstration, with no real customer funds or private keys.

Feature	Purpose
Spot trading	Submit and cancel selected limit orders, match orders and view their status.
Balances and ledger	Track available and reserved balances and record trade-related financial movements.
Market data	Show prices, order-book updates and recent trades.
AI Trading Guardian	Detect unusual activity, create risk alerts and support human review.
Safety guardrails	Apply order limits, balance checks, idempotency and controls to prevent duplicate or unauthorized actions.
Dynamic hot-wallet management	Simulate separate wallet compartments per asset and network, with liquidity targets, caps and controlled approvals.
Monitoring	Use Prometheus, Grafana and Alertmanager to monitor performance, failures, trading health and risk.
Account and device security	MFA, 60-second interactive inactivity lock, device loss response and risk-based transaction checks.

2 Scope and boundaries

In scope for architecture and simulated pilot

- Simulated MFA, 60-second interactive lock, stolen-device reporting, server-side revocation and transaction step-up; no real funds or credentials.
- Spot limit orders, cancellation, matching for selected markets, order history, balance reservation, basic market data and simulated settlement.
- AWS EKS with multi AZ worker nodes, ingress, Aurora PostgreSQL, cache, queues, GitLab CI/CD, Helm or GitOps and CloudFormation.
- Metrics with Prometheus, Grafana and Alertmanager; API, matching, wallet exposure, queue lag, security and ledger monitoring.
- Guardian rules, synthetic behavioral signals, review queue and simulated pause controls; optional model evaluated in shadow mode.

- Simulated or testnet deposits and withdrawals, isolated asset/network wallet pools, liquidity calculation, mocked signer and human approved transfer workflow.
- Cross Region backup copies, infrastructure recreation, recovery runbook, data reconciliation and game day tests.

- Explicitly out of scope for pilot
- Live customer funds, real private keys, production custody, direct connection to production exchanges or automatic on chain treasury transfers.
- Margin, leverage, derivatives, fiat rails, KYC/AML production operations, live custody integrations, token listing, complex market surveillance and customer support. Guaranteed fraud prevention, zero loss, global sub 100 ms latency, regulatory compliance certification or a production ready exchange.
- Production deployments, 24/7 support, independent security audit and live custody launch require a separately approved scope.

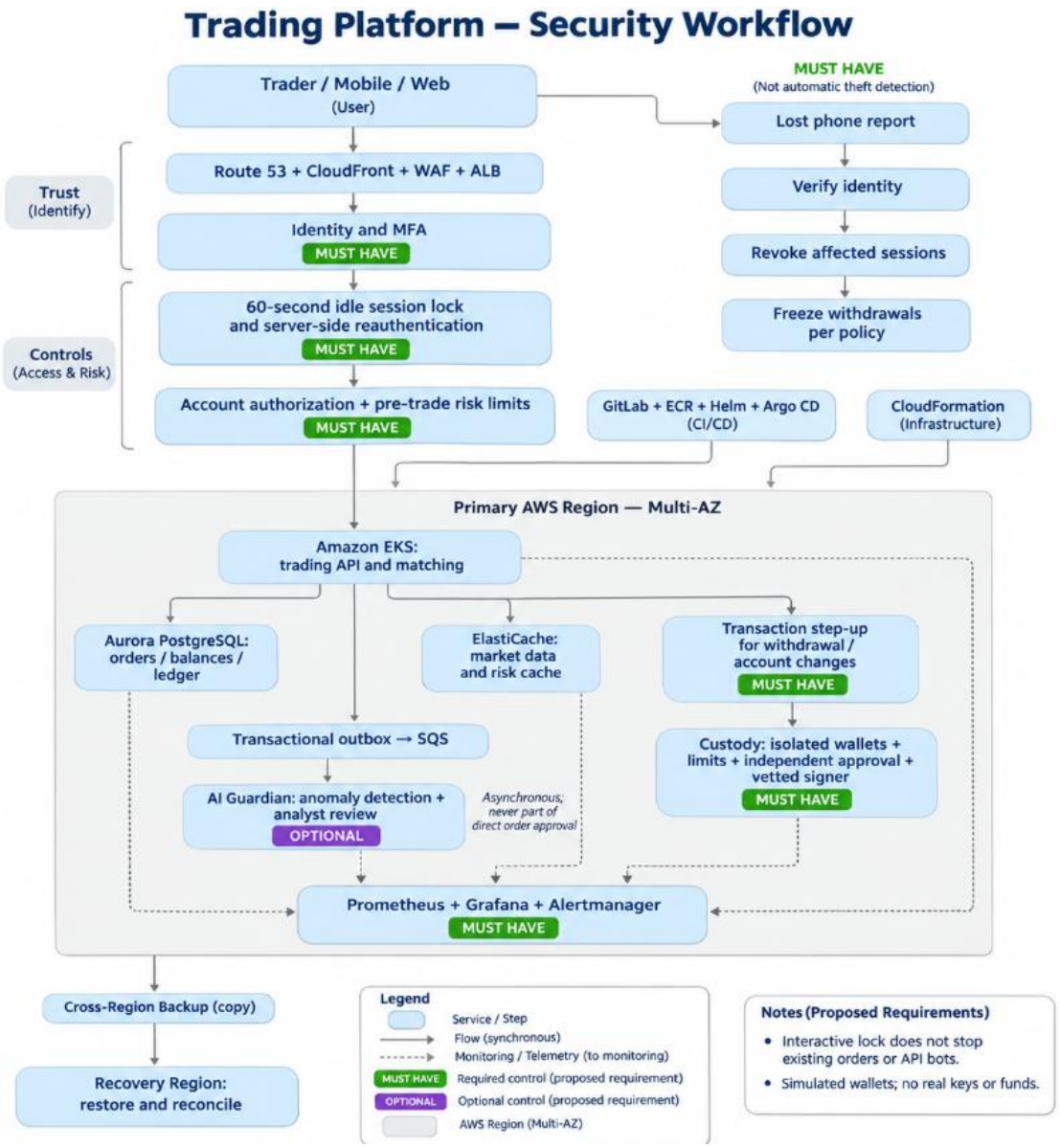
3 Proposed solution on AWS

Trading and platform requirements

ID	Requirement and output
TR-01	Authenticate users and authorize market, account and order access.
TR-02	Validate order size and price increments; enforce configured limits and reserve available balance atomically.
TR-03	Durably record the order and unique client id; return ACCEPTED only after commit, not after execution.
TR-04	Process accepted commands in defined market order, prevent dual ownership with fencing, and support cancel and partial fill states.
TR-05	Record trades and balanced ledger entries transactionally; replay committed state after failures and reconcile.
TR-06	Publish a reliable outbox event for downstream read models and Guardian; consumers handle duplicate events.

ID	Requirement and output
TR-07	Provide market data and order history with clearly identified freshness; never treat the cache as the ledger.
TR-08	Record audit evidence: actor, action, policy version, correlation id and decision. Protect sensitive data.

Architecture overview : apps Worklow with Added Security Layers



Detailed AWS architecture with added Security layers

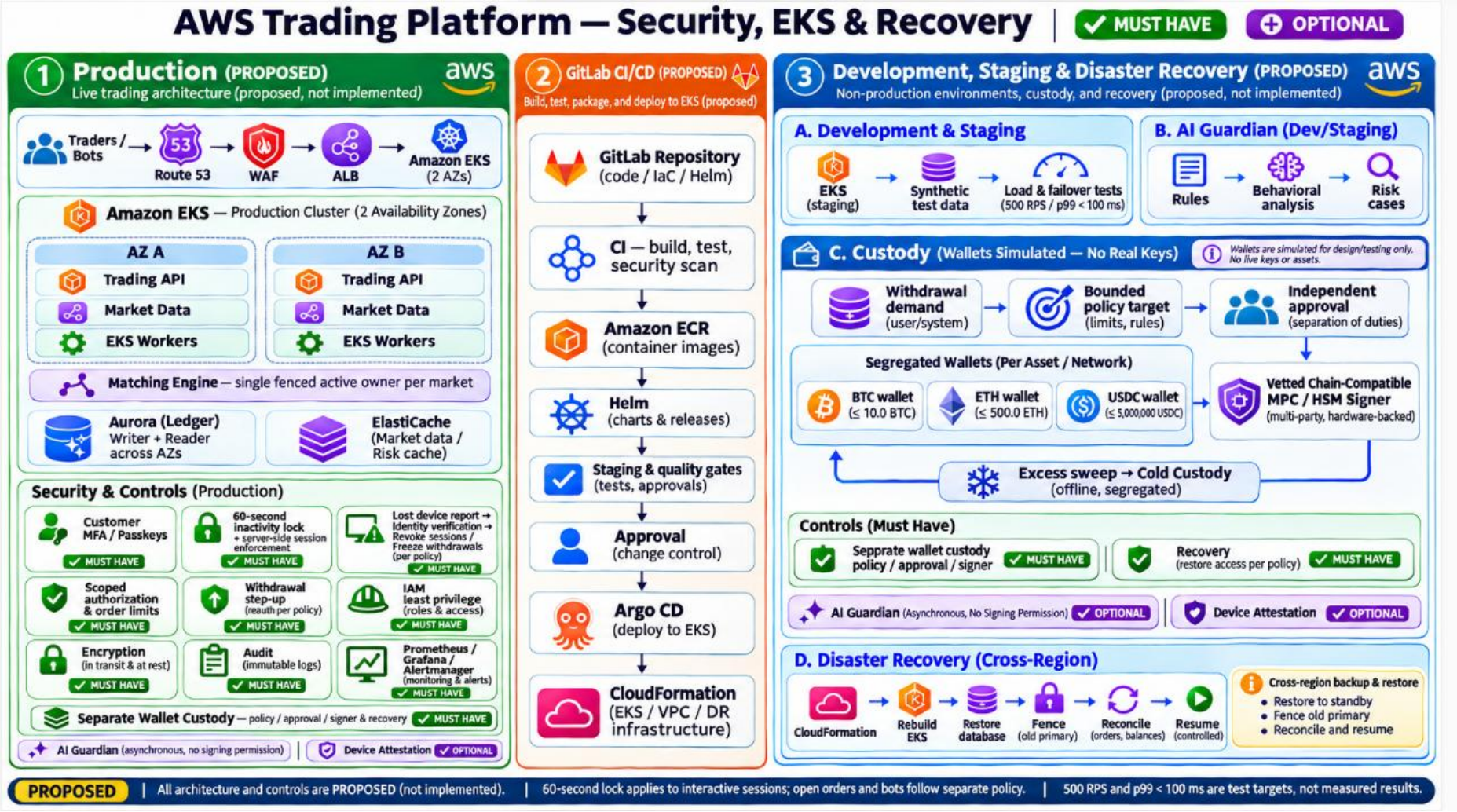


Diagram notes: wallet caps are illustrative placeholders, not recommended limits. The custody flow is simulated; fence the former primary before enabling recovery-region writes. 500 RPS and p99 are targets, not measured results.

AWS services and alternatives

Function	Selected option	Benefit / tradeoff	Alternative	Benefit / tradeoff
DNS	Route 53	Managed DNS / TTL affects failover	Route 53 + Global Accelerator	Static IPs, faster regional routing / extra cost
Content delivery	CloudFront	Edge caching / configuration and transfer cost	Direct S3 or ALB	Simpler / no edge caching
Web protection	WAF on CloudFront	Filters at the edge / rules need tuning	WAF on ALB	Regional protection / traffic reaches AWS Region first
API entry	Application Load Balancer	HTTP and WebSocket routing / API controls need design	API Gateway	Managed API controls / extra cost and latency to test
Customer identity	Cognito	Managed authentication / integration constraints	External identity provider	Flexible features / vendor and integration costs
Containers	EKS	Kubernetes ecosystem / cluster operations	ECS	Simpler orchestration / less Kubernetes portability; performance depends on workload.
Compute	EKS managed EC2 nodes	Node control / capacity and patching	EKS on Fargate	Less node management / less host control
Database	Aurora PostgreSQL	Transactions and failover / cost and performance testing	RDS PostgreSQL Multi-AZ	Managed PostgreSQL / different scaling and failover

Function	Selected option	Benefit / tradeoff	Alternative	Benefit / tradeoff
Cache	ElastiCache	Fast reads / cache consistency	DynamoDB	Managed key-value store / different access patterns
Events	SQS	Managed queues / limited replay	MSK	Streaming and replay / more complexity
Container registry	ECR	AWS-integrated / storage and replication costs	GitLab Registry	Integrated with GitLab / GitLab dependency
Monitoring	Prometheus + Grafana on EKS	Full control / team manages the stack	AWS Managed Prometheus + Grafana	Less administration / service charges
Infrastructure as code	CloudFormation	AWS-native deployment / template maintenance	AWS CDK	Reusable code / extra tooling; uses CloudFormation
AZ recovery	Multi-AZ EKS + Aurora	AZ resilience / higher capacity cost	Single AZ	Lower cost / AZ failure causes interruption
Regional DR	Cross-Region backup and restore	Lower cost / slower recovery and possible data loss	Aurora Global Database + pilot light	Faster recovery / higher cost; data loss still possible
Wallet custody	Vetted custody provider or MPC/HSM design	Controlled signing / provider evaluation	Custom CloudHSM-based custody	More control / greater security and operational responsibility

The table below keeps your original infrastructure choices and adds the security services needed for customer protection, custody and operations.

Priority: Must = required for the proposed production design; **Optional** = enhancement to evaluate later. Custody remains simulated for the MVP.

Function	Selected option	Benefit / tradeoff	Alternative	Benefit / tradeoff
DNS	Route 53	Managed DNS / TTL affects failover	Route 53 + Global Accelerator	Static IPs and regional routing / extra cost
Content delivery	CloudFront	Edge caching / configuration and transfer costs	Direct S3 or ALB	Simpler / no edge caching
Web protection	AWS WAF on CloudFront	Edge filtering / rules need tuning	AWS WAF on ALB	Regional protection / traffic reaches Region first
API entry	Application Load Balancer	HTTP and WebSocket routing / API controls need design	API Gateway	Managed API controls / extra cost and latency to test
Customer identity Must	Amazon Cognito	Managed authentication and MFA / integration constraints	Auth0 or Okta Customer Identity	More identity features / vendor cost and dependency
Session lock Must	Application session service on EKS	Enforces 60-second inactivity policy / custom implementation	External identity platform + session service	Managed identity features / still requires application enforcement
Lost-device response Must	Cognito revocation + application session store	Revoke sessions and restrict withdrawals / requires verified reporting and server-side checks	External identity provider + session store	Vendor tooling / integration and cost
Authorization Must	EKS authorization service + IAM	Scoped customer and workload permissions / policies need testing	Amazon Verified Permissions	Managed application authorization / added integration and policy complexity
Withdrawal	Application policy engine	Step-up authentication, limits	Custody-provider policy	Specialist controls / vendor

Function	Selected option	Benefit / tradeoff	Alternative	Benefit / tradeoff
protection Must	+ independent approval	and allowlists / requires workflow and custody integration	engine	and chain coverage must be evaluated
Wallet custody Must	Vetted custody provider or chain-compatible MPC/HSM design	Independent signing and compartment controls / provider evaluation	Custom AWS CloudHSM-based custody	Greater control / much greater operational and security responsibility
Workload identity Must	IAM roles for EKS workloads	Short-lived, scoped AWS credentials / role policies require maintenance	Static AWS access keys	Simple initially / credential exposure and rotation risk
Secrets Must	AWS Secrets Manager	Managed secret storage and rotation support / service cost	HashiCorp Vault	Flexible across platforms / operational complexity
Encryption Must	AWS KMS + TLS	Managed encryption keys and secure transport / key policies need care	External HSM or key management	Additional control / cost and integration
Audit Must	CloudTrail + protected S3 logs	Records AWS activity / application and custody events need separate audit logs	External SIEM, such as Splunk	Centralized investigation / licensing and integration costs
Threat detection Must	GuardDuty + operational alerting	AWS and EKS threat findings / coverage and alert tuning required	External detection platform	Broader cross-cloud visibility / additional cost
AI Guardian Optional	Guardian workers on EKS	Behavioral alerts and human review / model tuning and false positives	Third-party fraud platform	Specialist capabilities / vendor cost and data-sharing review
Device attestation Optional	Platform attestation + application checks	Additional device-risk signals / platform-specific integration	Third-party mobile security SDK	Managed tooling / SDK cost and vendor dependency
Containers	Amazon EKS	Kubernetes ecosystem / cluster	Amazon ECS	Simpler orchestration / less

Function	Selected option	Benefit / tradeoff	Alternative	Benefit / tradeoff
		operations		Kubernetes portability
Compute	EKS managed EC2 nodes	Node control / capacity and patching	EKS on Fargate	Less node management / less host control
Database	Aurora PostgreSQL	Transactions and failover / cost and performance testing	RDS PostgreSQL Multi-AZ	Managed PostgreSQL / different scaling and failover
Cache	ElastiCache	Fast reads / cache consistency work	DynamoDB	Managed key-value store / different access patterns
Events	Amazon SQS	Managed queues / limited replay	Amazon MSK	Streaming and replay / more complexity
Container registry	Amazon ECR	AWS integration / storage and replication costs	GitLab Registry	GitLab integration / dependency on GitLab
CI/CD security Must	GitLab scanning + ECR image scanning	Checks code and images / findings need triage	GitHub Advanced Security + ECR	Integrated GitHub workflow / changes the selected CI/CD platform
Monitoring	Prometheus, Grafana and Alertmanager on EKS	Full control / team manages the stack	Amazon Managed Service for Prometheus + Managed Grafana	Less administration / service charges
Infrastructure as code	CloudFormation	AWS-native deployment / template maintenance	AWS CDK	Reusable code / extra tooling; deploys through CloudFormation
AZ recovery	Multi-AZ EKS + Aurora	AZ resilience / higher capacity cost	Single AZ	Lower cost / AZ failure interrupts service
Regional DR	Cross-Region backup and restore	Lower cost / slower recovery and possible data loss	Aurora Global Database + pilot light	Faster recovery / higher cost; data loss still possible

Implementation notes:

- The **60-second lock** applies to interactive sessions. Existing orders and API bots follow separate policies. The app and backend must enforce this; Cognito alone does not provide the complete inactivity control.
- **Revocation needs a server-side session check:** a revoked Cognito token may still pass a local JWT signature-and-expiry check. Amazon Cognito
- A **passkey can satisfy MFA** when configured with user verification and supported by the chosen Cognito flow. Amazon Cognito
- Guardian analyzes events asynchronously; it has **no access to signing keys or permission to transfer funds**.

4 Security and safety architecture

Account Security Layers and priorities

Priority 1 is protecting user access and preventing unauthorized account actions. Priority 2 is protecting exchange funds and the ledger; priority 3 is availability, monitoring and recovery. AI-supported detection is a later enhancement built on deterministic controls. The challenge does not require a live exchange: the MVP uses simulated balances and wallets, while the must-have list below defines the proposed minimum before handling real customer assets.

Priority	Security layer	What we implement and why
Must have	Identity and MFA	Unique accounts; MFA for authenticated users. Prefer phishing-resistant passkeys or security keys with user verification; provide tested recovery and accessible fallback. Protect privileged administrators separately.
Must have	Interactive session and device lock	Lock the trading interface after 60 seconds of no user interaction and on detected device lock. Reauthenticate for account actions; enforce the locked state on the backend.
Must have	Lost or stolen device	Provide a verified account recovery path and device/session revocation; immediately block new sensitive requests and withdrawal requests for the reported device/account according to the selected freeze scope.
Must have	Transaction and account changes	Require fresh, phishing-resistant step-up for withdrawals, new withdrawal destinations, MFA changes and recovery; bind confirmation to transaction details. Use limits, alerts and controlled withdrawal holds.
Must have	Secure account recovery	Offer recovery without depending only on the missing phone; verify identity through approved methods, notify the user, limit recovery attempts and require separate approval for sensitive changes.

Priority	Security layer	What we implement and why
Must have	Authorization and API protection	Enforce authorization server-side per account and action; apply least privilege, rate limits, replay prevention and idempotency. Manage API keys separately with scoped permissions and revocation.
Must have	Hot wallet and custody	Separate asset/network compartments and independent signing boundaries; set approved caps, allowlists, multi-person approval and reconciliation. The pilot simulates custody only.
Must have	Platform and data	TLS, encryption at rest, KMS-managed service keys, secrets management, EKS workload IAM, network isolation, patched images and dependency scans. Chain-compatible custody signing needs a vetted provider or separately validated HSM/MPC design.
Must have	Ledger, monitoring and recovery	Protect the authoritative transactional ledger, reconcile activity, maintain audit trails and alerts, and test multi-AZ failure plus cross-Region backups and restore.
Nice to have	Adaptive analysis and Guardian	Add behavioral anomaly detection, analyst case summaries and selective step-up after validated thresholds and shadow-mode evaluation; never let an LLM sign transactions.
Nice to have	Extra device and withdrawal controls	Offer trusted-device management, device attestation as a risk signal, configurable withdrawal delays and additional security keys. Avoid treating location or high volume alone as proof of fraud.

60-second timeout and user experience

Recommended interpretation of the requested 60-second timeout: after 60 seconds without meaningful user interaction, the interactive trading session becomes locked; private screens are hidden and the server denies new sensitive requests until the user reauthenticates. This is an application lock and server-side authorization state, not necessarily a full account logout or token deletion. Public market prices may remain visible without private account information if product policy permits.

- Count deliberate touch, keyboard, mouse or authenticated user actions. Automatic price updates, WebSocket pings and background refresh do not reset the inactivity clock. Replay or stolen-token activity may imitate user interaction; the idle timer alone cannot stop that, so combine it with device-bound authentication, revocation and risk monitoring.
- On mobile app backgrounding, immediately cover sensitive content; lock when the OS reports device lock or protected data becomes unavailable, and always re-check elapsed time and device state on resume. Platform capabilities vary; a web SPA cannot reliably detect every screen lock. Treat unavailable or uncertain state conservatively, without equating an OS lock-state signal with authentication.

- On unlock, require a fresh server-verified WebAuthn/passkey challenge or another approved reauthentication; local biometric-only UI unlock does not by itself reactivate a server-locked session. When supported, face/fingerprint unlocks the device-bound key without sharing biometric data with the service. Provide accessible alternatives.
- Users actively interacting do not encounter the idle lock. A user-initiated high-risk action still requires step-up based on policy. Backend controls apply to every protected API request and WebSocket action; passive activity cannot extend the timer. Clearly distinguish a 60-second interactive lock from token expiry and any separate absolute session lifetime.
- A session lock does not automatically cancel open orders, stop bots or cancel withdrawals. Explain the behavior in the UI. Offer explicit, authorized controls for canceling orders or freezing the account; recovery and loss policies must document pending and in-flight operations.

Mobile reported lost or stolen

The exchange does not automatically know that a phone was stolen. A report must come from the customer through a verified secure web session, a previously enrolled recovery method or a trained support team with an established recovery procedure. OS remote-lock or device-management integrations are optional and cannot be assumed reliable or instantaneous.

- 1. Customer reports the device and chooses the scope: revoke the device or freeze the account. Verify the request without requiring access to the missing phone; provide a monitored support route.
- 2. Atomically mark affected device and sessions revoked in the authoritative session store. Revoke refresh credentials, sever or revalidate WebSocket sessions, reject protected actions server-side and invalidate device-bound trust. Notify the customer through an independent enrolled channel.
- 3. Freeze new withdrawals for the selected account or as required by risk policy. Pause or review pending withdrawals if still stoppable; on-chain transactions already broadcast cannot be guaranteed reversible. Account-wide restrictions and API-key revocation require the documented scope; if broader compromise is suspected, provide an immediate account-wide freeze.
- 4. Present existing open orders, API keys and pending withdrawals clearly. Do not silently cancel open orders or disable unrelated API clients; obtain authorization or apply a disclosed emergency policy.
- 5. Restore access through verified recovery, require new passkey/MFA enrollment and apply documented protection to credential changes and withdrawal destinations. Log the action and review suspicious events.

Revocation must not rely only on a browser lock, token refresh or deleting a local token. A valid self-contained access token may otherwise remain usable until expiry. Use server-side session/version or revocation checks at the API and WebSocket authorization boundary, with bounded cache propagation; reject sensitive actions when revocation status cannot be verified. Measure and document the worst-case revocation delay and in-flight request behavior.

Risk-based security without unnecessary friction

Action	Protection	User experience
View public markets	No private-session access needed.	No login required where permitted.
Access account or ordinary trade	MFA at sign-in; 60-second server-enforced idle lock; quick passkey-based reauthentication.	Active users continue; idle users unlock with a short biometric prompt where supported.

Action	Protection	User experience
Withdraw, add destination or change MFA	Fresh phishing-resistant step-up and transaction-specific confirmation; limits and optional policy-based delay.	Extra prompt only for higher-risk actions.
New device, unusual behavior or loss report	Risk-based alerts and proportionate step-up or freeze; human review when needed.	Explain the reason and provide recovery; no claim that one signal proves fraud.
API trading bots	Separate scoped API credentials, expiry and optional IP restrictions; explicit revocation.	No 60-second human UI-lock timer for bots; apply dedicated server policies and trade safeguards.

MVP and production release gates

MVP demonstration: synthetic accounts, MFA and server-enforced lock simulation, device-loss and revocation workflow, transaction guardrail scenarios, simulated custody and audit trail. Validate with test accounts; no live funds, private keys or custody claims. User experience should be tested with active traders and accessibility needs before policy is finalized.

Before any real-money launch, do not ship without strong account authentication and recovery, enforced session/device revocation, authorization and step-up, ledger correctness, independent custody controls, withdrawal safeguards, logging and monitoring, backups and recovery tests, incident support and independent security review. Regulatory and custody obligations require separate specialist assessment. Nice-to-have AI and device attestation are not substitutes for these controls.

Security acceptance tests and metrics

- At 59 seconds idle a session remains usable; at 60 seconds sensitive API and WebSocket actions are denied until server-verified reauthentication. Test clock skew, concurrency, sleep/resume, offline mode and no timer reset by heartbeats.
- Device lock/background and unlock behavior follows the supported iOS/Android lifecycle; on resume always revalidate. Test web browsers separately and never assume reliable screen-lock detection.
- A confirmed device-loss report blocks affected sessions, rejects old tokens and terminates or reauthorizes active WebSocket sessions within the documented revocation objective; test cache staleness and authorization-store outage.
- Account freeze blocks new withdrawals and follows the defined policy for pending and already broadcast transfers. Existing orders and bots remain governed by the disclosed chosen scope. Recovery works without the lost phone and triggers notifications.
- Step-up cannot approve a modified transaction or be replayed; balances and private data are hidden when locked. Authorized API clients follow their own policy. Test accessibility, passkey fallback, support recovery and audit events.
- Measure time to unlock, authentication success and abandonment, false lockouts, MFA recovery time, revocation latency, failed logins, withdrawal holds, customer support cases and p95/p99 added latency. Performance target is not proven.

Reference guidance: NIST Digital Identity Guidelines https://pages.nist.gov/800-63-4/sp800-63b.html ; OWASP mobile security guidance https://cheatsheetseries.owasp.org/cheatsheets/Mobile_Application_Security_Cheat_Sheet.html . Apply the selected policies to the actual platform and verify current service and platform capabilities.

Custody risk and human oversight

The proposed controls address custody risk. Binance reported a major wallet breach in October 2026; the exact attack path and effectiveness of these proposed controls require independent investigation. Source: https://www.reuters.com/legal/government/gone-20-minutes-how-biggest-heist-crypto-history-unfolded-2026-10-30/

AI Guardian and human control

Fast deterministic rules act before order acceptance. The Guardian consumes committed events asynchronously, identifies suspicious patterns and creates explainable risk cases. It may propose or apply preapproved account level restrictions through a separate policy service; it cannot sign or transfer assets.

Signal	Illustrative response
Unusual order velocity	Compare with account profile, authorized bot limits and market conditions; alert or hold according to policy.
High value plus new device	Combine signals and request review or step up authentication where appropriate.
Repeated cancellations or abnormal patterns	Create a surveillance case; legitimate market making is not automatically treated as fraud.
Guardian outage or lag	Alert operators; use approved deterministic fallback and documented fail open or fail closed rules.

Pilot acceptance: test synthetic normal and suspicious scenarios, document false positives and delayed detection, review every high impact action, and log policy/model version. An ML model is optional and starts in shadow mode; no effectiveness claim is made without evaluation.

Dynamic compartmentalized hot wallet design

Objective: keep only the liquidity needed for expected near term withdrawals plus approved buffer in each hot wallet, subject to asset/network exposure caps. The system responds to withdrawal demand, on chain congestion and treasury policy. Trading volume may inform a forecast, but it is not an obligation to move funds on chain.

Asset and network compartments

Compartment	Policy and operational note
BTC on Bitcoin	Dedicated hot wallet and independent exposure cap; account for UTXO management, confirmations and transaction fees.
ETH on Ethereum	Dedicated hot wallet and cap; reserve native ETH for network fees.
USDC on Ethereum	Separate token accounting and exposure limit; ERC 20 transactions also consume ETH gas. Distinct addresses alone do not ensure key isolation.
Other assets / networks	Onboard only after asset, network, signer, custody, risk and reconciliation reviews. Same symbol on different networks requires separate treatment.

Dynamic hot-wallet controls

- **Isolate wallets:** Use separate signing keys, permissions and limits per asset and network. Segmentation reduces exposure but cannot eliminate losses.
- **Size dynamically:** Set each target balance from forecast withdrawals, fees and a buffer, within approved minimum and maximum limits. Include pending transfers.
- **Manage liquidity:** Request a top-up when funds are low; sweep sustained excess to cold custody. Apply cooldowns and transfer limits.
- **Control transfers:** Require independent approval, destination allowlists and signer-level policy checks. Pause on suspicious activity, stale data or reconciliation errors.
- **Protect keys:** Use a vetted custody provider or chain-compatible HSM/MPC signer. Never expose keys to AI, logs or CI.
- **Reconcile:** Compare blockchain balances, pending transfers, custody records and the internal ledger. Investigate differences before resuming withdrawals.
- **Pilot:** Simulate recommendations and approvals with **no real funds or keys**.

Illustrative example: A wallet holds 0.40 BTC, with 0.10 BTC reliably in transit and counted under policy. For a target of 0.85 BTC, the proposed top-up is 0.35 BTC; otherwise, hold and investigate.

5 Availability performance monitoring and disaster recovery

Requirement	Approach and evidence
Availability	EKS replicas and nodes across AZs; scheduling and disruption controls; spare capacity for AZ loss; Aurora reader and matching standby. Test node and AZ failure.
Performance	500 requests/sec mixed API workload and p99 <100 ms measured end to end within an agreed geography. Include fast guardrail checks; test wallet and WebSocket paths separately. Measure latency, errors and saturation.
Operations	Prometheus and Grafana dashboards and Alertmanager alerts: p99, errors, order backlog, queue age, ledger discrepancies, hot balances, caps, gas, reconciliation and Guardian lag. Monitoring has its own availability plan.
Regional DR	Cross Region backup and restore; redeploy EKS and apps via CloudFormation and GitOps; fence old primary; restore, reconcile and approve before restart. Snapshot RPO can be nonzero; recent records may be unavailable.
Security	IAM least privilege, network isolation, encryption, scoped identities, secrets management, image and dependency scanning, audit logging, tested incident response.

6 Scaling plan

- API and WebSocket: scale pods and EC2 nodes independently using request latency, CPU, connection counts and resource headroom.
- Matching and events: assign separate markets to workers, retain one fenced active owner per market, and scale Guardian consumers using queue age. A hot market needs its own capacity tests.
- Data: cache suitable reads; optimize transactions and scale the Aurora writer vertically before considering partitioning. Readers can lag and must not be assumed to provide immediate balance consistency.
- Availability: maintain or provision capacity in healthy AZs. Test both the 500 RPS baseline and the required degraded-mode behavior after AZ failure.

7 Starting size and indicative cost

Component	Recommended starting size	Why / how to scale	CAPEX	Estimated OPEX / month
AWS Region	Singapore, 2 AZs	Regional placement and resilience	$0	No separate Region fee; resources charged below
EKS cluster	1 production cluster	Kubernetes control plane; scale workloads on nodes	$0	~$73
EKS worker nodes	4 × m7i.xlarge, 2 per AZ	Add nodes or use larger instances after testing	$0	~$736
Order API	2 pods across AZs	Add pods as traffic grows	$0	Included in worker nodes
Market data / WebSocket	2 pods across AZs	Scale by connection and message volume	$0	Included in worker nodes
Matching engine	1 active + 1 standby per market	Scale different markets across workers	$0	Included in worker nodes
AI Guardian	2 small worker pods	Scale from queue backlog	$0	Included in worker nodes; external AI services extra

Component	Recommended starting size	Why / how to scale	CAPEX	Estimated OPEX / month
Aurora PostgreSQL	1 writer + 1 reader, db.r7g.large	Increase writer size when needed	$0	~$486 for instances; storage, I/O and backups extra
ElastiCache	1 primary + 1 replica, example cache.t4g.small Valkey	Resize after measuring cache usage	$0	~$56 for nodes; other charges extra
SQS	Queues for initial event flows	Scale consumers as backlog grows	$0	Usage-based; estimate once message volume is known
Monitoring	Prometheus, Grafana and Alertmanager on EKS	Add capacity as metrics and retention grow	$0	Compute included in nodes; storage and additional capacity extra
Regional DR	Cross-Region backups and recovery templates	Rebuild the platform after a Regional disaster	$0	Backup storage and copy charges; recovery resources extra when activated
Edge and networking	Route 53, CloudFront, WAF, ALB, NAT and data transfer	Scale with requests, traffic and connections	$0	Usage-based; estimate separately

Budget basis: approximately $1,351/month for EKS standard control plane, four On-Demand m7i.xlarge nodes, two Aurora db.r7g.large instances and two cache.t4g.small Valkey nodes, at 730 hours in Singapore. This is a partial estimate, not total cost. Storage, I/O, networking, edge, logs, backup copies, staging, support, taxes and custody are additional; confirm current prices in AWS Pricing Calculator. $0 CAPEX means no cloud hardware purchase, not zero engineering cost. Sizing and performance remain unverified.

Security costs: identity provider, passkey/recovery services, security logs, monitoring, support, testing and custody controls are not included in the partial MVP cost estimate.

8. Delivery phases and outputs

Estimates are illustrative and assume a small team, access to AWS accounts and timely business input. Phases can overlap. The MVP uses **simulated funds and signing**.

Phase	Indicative duration	Output
0. Discovery	1–2 weeks	Confirm requirements, traffic assumptions, security priorities, threat model, custody boundaries, architecture and test plan.
1. Trading and security foundation	2–3 weeks	Simulated orders and ledger; Multi-AZ EKS; CI/CD; MFA, authorization, 60-second idle lock, encryption, audit logs and basic dashboards.
2. Guardian and wallet simulator	2–3 weeks	Rule-based Guardian; simulated wallet compartments, sizing, limits, approvals, transaction step-up, reconciliation and lost-device response.
3. Performance and cost evaluation	1–2 weeks	Test 500 RPS and p99 <100 ms, including security controls; identify bottlenecks, measure AWS costs and recommend sizing changes.
4. Scaling	1–2 weeks	Scale EKS pods and nodes, database and event workers; test higher traffic and document limits, security impacts and costs.
5. Resilience and validation	1–2 weeks	Test AZ failure, Regional recovery, session revocation, custody safeguards and security incidents; deliver evidence and runbooks.
6. Production readiness	Separate scope	Independent security and custody review; real-asset controls, penetration testing, operating model and launch decision.

9. Acceptance criteria and test evidence

Area	Acceptance criteria
Trading and ledger	Supported orders process correctly; ledger entries balance; retries do not create duplicate logical orders or trades.
Performance	Test 500 RPS with p99 <100 ms under the agreed workload, client geography, duration and error threshold; report results and bottlenecks.
Account protection	Test MFA, server-enforced 60-second inactivity lock, reauthentication, lost-device reporting and session revocation. Confirm the policy for open orders, API bots and pending withdrawals.
Authorization	Verify users, workloads and administrators can access only their authorized resources; denied actions are logged.
Withdrawal safeguards	Test step-up authentication, destination allowlists, transaction limits and independent approval using simulated transactions.
Wallets and custody	Simulated wallet targets respect asset/network limits, pending transfers, approvals and gas requirements. Test reconciliation and pause conditions.
Guardian	Synthetic risk cases produce traceable alerts and review outcomes. Guardian cannot access keys or sign transactions.
Availability	AZ and matching-owner failures do not cause duplicate trades; record interruption and recovery time.
Security and monitoring	Test security alerts, audit records, CI/CD checks, secrets handling and incident response; document failures and remediation.
Disaster recovery	Restore cross-Region backups; measure recovery time and data-loss exposure; fence the former primary, reconcile and obtain approval before resuming.
Handoff	Deliver diagrams, source code, test evidence, dashboards, runbooks and an outstanding-gap register.

10. Assumptions and risks

- **Performance:** 500 RPS and p99 <100 ms are targets, not measured results. Outcomes depend on the request mix, client geography, error threshold and infrastructure sizing.
- **Session security:** The 60-second lock applies to interactive sessions. Open orders and automated API clients follow separate policies. Lost-device protection starts when the device is reported; it is not automatically detected.
- **Token revocation:** Immediate enforcement requires server-side session checks; locally validating a token's signature and expiry alone is insufficient.
- **Availability and DR:** Multi-AZ supports recovery from an AZ outage. Cross-Region backups can omit recently accepted transactions, so recovery requires reconciliation.
- **Custody:** The MVP uses simulated funds and signer behavior. Segregated keys, permissions and approvals reduce exposure but cannot eliminate shared-control risk.
- **Guardian:** Automated risk analysis can produce false alerts or miss suspicious activity. Required transaction and custody controls remain effective without AI.
- **Cost:** Initial estimates exclude some variable and supporting charges. Validate costs and sizing through workload, security and failover tests.
- **Production launch:** Real-asset handling requires a separate security and custody review, approved operating procedures and a launch decision.